information

MDPI

Article

Activation-Guided Layer Selection for LoRA

Aditya Dawadikar [1], Pooja Shyamsundar [2], Rashmi Vishwanath Bhat [3] and Navrati Saxena [1,*]

[1] Department of Computer Science, San Jose State University, San Jose, CA 95192, USA; aditya.dawadikar@sjsu.edu
[2] IBM, Armonk, NY 10504, USA; poojashyam2k@gmail.com
[3] Salesforce, San Francisco, CA 94105, USA; rashmi.vishwanathbhat@gmail.com
* Correspondence: navrati.saxena@sjsu.edu

Abstract

Low-Rank Adaptation (LoRA) has become a widely adopted parameter-efficient fine-tuning (PEFT) technique for large language models (LLMs). LoRA's benefits stem from its light weight and modular adapters. Standard LoRA applies adapters uniformly across all Transformer layers, implicitly assuming that each layer contributes equally to task adaptation. However, LLMs are found to have internal substructures that contribute in a disproportionate manner. In this work, we provide a theoretical analysis of how LoRA weight updates are influenced by a layer's activation magnitude. We propose Act-LoRA, a simple activation-guided layer selection strategy for selective Low-Rank Adaptation. We evaluate this strategy for both encoder-only and decoder-only architectures using the GLUE benchmark. Our method achieved a 20% GPUh saving with a 1% drop in GLUE score using DeBERTaV3-Base on a single-instance GPU with 50% less LoRA parameters. It also achieved 2% GPUh savings with a less than 0.15% drop in GLUE score with the Llama-3.1-8B model in Distributed Data Parallel mode with 25% fewer LoRA parameters. Our experiments and analysis show that the compute and memory requirements of LoRA adapters increase linearly with the number of selected layers. We further compare activation-guided selection against gradient-guided importance metrics and show that activation norms yield more stable and reproducible layer rankings across seeds and datasets. Overall, our results demonstrate that activation-guided layer selection is a practical and effective way to improve the efficiency of LoRA fine-tuning, making it immediately compatible with some existing PEFT techniques and distributed training frameworks.

Keywords: LoRA; PEFT; activation norms; gradient norms; LLM

Check for updates

Academic Editors: Tania Yankova and Galina Ilieva

Received: 17 January 2026
Revised: 24 February 2026
Accepted: 4 March 2026
Published: 12 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Large language models (LLMs) have consistently pushed the boundaries of natural language processing and reasoning. Since the advent of the Transformer architecture [1,2], a wide range of model families have been proposed and adopted at scale. As LLMs benefit from richer contextual representations, increasing sequence length and model depth substantially amplifies computational and memory demands, often growing superlinearly with respect to input size. As a result, adapting pretrained models to downstream tasks has become a practical necessity, as training task-specific models from scratch at this scale is prohibitively expensive. Knowledge Distillation is a technique used to train a smaller model from larger teacher model. In ref. [3], Distill-BERT was able to retain 97% of its parent BERT model's language comprehension capability, ref. [4]. Another parallel line of research is the

Parameter-efficient fine-tuning (PEFT) methods, which address this challenge by updating only a small fraction of model parameters while preserving most of the pretrained weights.

A variety of PEFT techniques have since been proposed, broadly falling into adapter-based approaches (e.g., Adapter Modules [5], LoRA [6], IA^3—Infused Adapter by Inhibiting and Amplifying Inner Activations [7]) and prompt-based approaches (e.g., prompt tuning [8] and prefix tuning [9]). These methods aim to adapt large pretrained models to specific tasks at a fraction of the cost of full fine-tuning. In this work, we focus on Low-Rank Adaptation (LoRA), one of the most widely adopted PEFT techniques due to its simplicity, modularity, and strong empirical performance.

LoRA injects low-rank update matrices into existing linear projections by decomposing weight updates into two trainable matrices, while keeping the original model weights frozen. This non-destructive formulation allows adapters to be added or removed without modifying the backbone model, enabling a single pretrained network to support multiple tasks by swapping lightweight adapters at inference time. In practice, LoRA typically introduces only 1–2% additional trainable parameters, substantially reducing the training cost and memory requirements compared to full fine-tuning.

Despite its effectiveness, standard LoRA applies adapters uniformly across all Transformer layers, implicitly assuming that each layer contributes equally to downstream adaptation. In practice, this assumption is rarely valid: prior work on representation analysis and layer-wise probing suggests that task-relevant information is often concentrated in a subset of layers [10–12]. Uniformly allocating adaptation capacity across all layers can therefore lead to redundant computation and the inefficient use of parameters [13,14]. While large-scale models benefit from techniques such as structured layer pruning, knowledge distillation, and layer skipping, these approaches are typically applied post hoc—after training a dense parent model to convergence—and do not reduce the compute already expended during fine-tuning.

Recent work on neural scaling laws has shown that model performance improves in a highly non-linear manner with respect to compute, model capacity, and data [15]. When the training dataset is held constant, increasing compute—typically by scaling model size or training duration—yields diminishing returns, such that incremental gains in accuracy require a disproportionately large increase in compute. Conversely, for a fixed compute budget, increasing the amount of training data has been shown to reduce loss more effectively than increasing model capacity alone. This imbalance between compute, data, and model size leads many models to operate in compute-inefficient regimes, as further emphasized by Hoffmann et al. [16].

Together, these findings suggest that naively scaling compute is not an efficient strategy for improving downstream performance, and instead motivate approaches that allocate adaptation capacity selectively, enabling competitive performances under constrained compute and data budgets. In this work, we propose an activation-guided layer selection strategy for LoRA that identifies a small set of influential layers and restricts adapter insertion to only those layers. Importantly, our approach does not modify the LoRA formulation; rather, it optimizes where LoRA adapters are applied, reducing redundancy while preserving downstream performance.

The use of methods like layer pruning for adaptive model depth is well established in the NLP and Deep Learning literature. In 2019, ref. [17] showed that while over-parameterized models often produce state-of-the-art results, they incur a substantial compute cost. Their work demonstrated that selecting sub-networks of varying depth can preserve downstream performance with minimal degradation while significantly reducing compute cost. Similarly in 2022, ref. [18] applied structured layer pruning to

BERT [4], RoBERTa [19] and XLNet, pruning up to 40% of the layers while preserving 98% of the original GLUE score.

Yao et al. in their 2024 paper [20] showed that selectively adapting a subset of the most influential layers can be beneficial to avoid sub-optimal fine-tuning results and improve performance. Their proposed method, IST (Importance-aware Sparse Tuning), is motivated by empirical observations based on ablations, resulting in a degraded model performance. IST employs Reinforcement Learning to identify the best structures.

Our work is motivated by a similar philosophy: concentrating adaptation capacity under a fixed parameter budget while retaining a maximal downstream performance and achieving considerable compute savings. However, our contribution differs fundamentally in mechanism. Rather than pruning layers from the backbone, or online RL-based importance computation, we exploit the structural properties of LLMs to selectively apply adaptation capacity to layers that are already aligned with the task. A detailed literature review summary is provided in Table 1.

Table 1. Overview of literature review.

Aspect	Concept	Citation
PEFT	IA3	[7]
	Prefix Tuning	[9]
	Prompt Tuning	[8]
	Adapter Modules	[5]
LoRA	Low-Rank Adaptation	[6]
	Adaptive rank allocation (AdaLoRA)	[13,21,22]
	Dynamic rank slicing (DyLoRA)	[23,24]
	Ablation-based rank importance (ALoRA)	[25]
	Meta-learned rank selection (AutoLoRA)	[26]
	LoRA-Drop	[27]
	Memory-efficient fine-tuning (QLoRA, GaLore, LoRA-FA)	[28–30]
	Structural adapter variants (Delta-LoRA, DoRA, VeRA, LoRA+, X-LoRA, Dynamic LoRA, NoRA, LoHa, LoKr)	[14,31–38]
Layer Specialization & Internal Substructure	Uniform Information Distribution in Layers	[20,39]
	Layer Task Specialization	[10,11,40,41]
	Layer Task Contribution Patterns	[12,42,43]
Layer Importance Metrics	Second-order (Hessian-based) methods	[44,45]
	Fisher-based sensitivity	[46]
	Gradient/Weight/Loss-based sensitivity metrics	[13,14,39]
	Activation-based importance	[10,20,36,47]

Research Questions

In this paper, we discuss the following research questions:

- (RQ-1) Is selectively adapting a subset of layers sufficient to retain a strong task performance?
- (RQ-2) Do activation norms provide a stable and informative signal for layer selection?

- (RQ-3) Is the activation norms-based importance score task-dependent?

2. Literature Review

2.1. Low-Rank Adaptation

In traditional full fine-tuning, all model weights are updated, which often yields a strong performance but incurs substantial storage and GPU overhead. Adapter-based approaches insert lightweight trainable modules between Transformer layers while keeping the backbone frozen [5]. IA3 scales internal activations using learned vectors to modulate attention and feed-forward components without introducing large new parameter blocks [38]. Prompt tuning optimizes soft input embeddings appended to the model input [8], whereas prefix tuning injects continuous task-specific vectors directly into the attention mechanism [9].

LoRA was first introduced by Hu et al. as a parameter-efficient fine-tuning method that applies low-rank decompositions to weight updates, substantially reducing the number of trainable parameters with minimal impact on downstream accuracy [6]. A key advantage of LoRA is its modular, non-destructive design, which allows task-specific adapters to be added or removed without modifying the pretrained backbone. By training only a small fraction of additional parameters, LoRA significantly reduces both the computational cost and memory requirements compared to full fine-tuning. Since its introduction, numerous extensions and variants of LoRA have been proposed.

AdaLoRA is a widely adopted extension of LoRA that adaptively reallocates parameter budgets across layers during fine-tuning [13]. It reparameterizes the low-rank weight update using an SVD-like formulation, $\Delta W = P\Lambda Q$, where P and Q approximate singular vectors and Λ contains trainable singular values. During training, AdaLoRA computes an importance score for each singular triplet based on a sensitivity metric derived from smoothed gradients and weights. A global budget scheduler progressively prunes less important singular values until a target rank is reached, enabling dynamic rank allocation. It intentionally overparameterizes LoRA adapters during early training and progressively prunes low-importance singular directions in the SVD parameter space to reach a target rank.

However, AdaLoRA's sensitivity based metric is largely heuristic and can be unstable due to gradient noise during training. Additionally, AdaLoRA evaluates parameters independently and does not explicitly consider the structural relationships between singular values in the SVD-based LoRA decomposition. These limitations can lead to suboptimal rank allocation and reduced efficiency. Recent work addresses these limitations by proposing more principled importance estimation strategies. SalientLoRA [21] introduce salience-based rank evaluation that considers singular value magnitudes, temporal stability, and inter-rank dependencies, enabling more reliable and efficient adaptive rank allocation. Bayesian approaches replace sensitivity with theoretically grounded metrics such as signal-to-noise ratio (SNR) [22] to better capture parameter relevance.

DyLoRA proposes a search-free mechanism for dynamic rank adjustment by overparameterizing a single LoRA adapter and training it using a mixture of rank slices at each iteration, with truncation inspired by nested dropout [23,24]. At inference time, different effective ranks can be obtained by activating contiguous subsets of the trained adapter, enabling rank flexibility without incurring adapter switch overhead.

ALoRA explores an explicit allocation strategy known as AB-LoRA (Ablation-Based LoRA) [25]. It introduces an SVD-like formulation in which a diagonal matrix controls rank contributions, aiming to avoid heuristic importance measures. The method evaluates the impact of each rank through an ablation-style analysis by constructing three model variants for a given rank r: (1) the complete model with all ranks present, (2) a model retaining only

rank r within a layer, (3) a model with all ranks except r. Importance scores are derived from performance differences across these variants, providing an empirical measure of rank significance. Importantly, this ablation-based evaluation offers a direct, model-agnostic notion of importance, which we later adopt as an empirical reference for validating layer importance metrics.

AutoLoRA leverages meta-learning to automatically determine rank configurations for LoRA adapters [26]. It adopts an SVD-like formulation in which the diagonal singular value matrix is replaced by a learnable weight vector, whose entries are optimized to emphasize or suppress rank components. A bi-level optimization procedure, similar to Model-Agnostic Meta-Learning (MAML), alternates between updating LoRA parameters on the training set and adjusting selection variables α on a validation set. After convergence, effective ranks are obtained by thresholding α, followed by retraining with the selected configuration. This enables data-driven, layer-wise rank adaptation without manual tuning.

In terms of memory efficiency, QLoRA combines LoRA with 4-bit quantization of the frozen base model, enabling the fine-tuning of large-scale language models on commodity GPUs by dramatically reducing the memory footprint of model weights [30]. By storing base parameters in 4-bit precision while maintaining higher-precision optimizer states and LoRA parameters, QLoRA achieves multi-fold reductions in memory usage without a significant degradation in downstream performance. This makes it possible to fine-tune models with billions of parameters under practical hardware constraints. In our experiments, we adopt QLoRA to enable fine-tuning of the LLaMA-3.1-8B model within the available GPU memory budget. Complementary approaches such as GaLore reduce gradient memory usage during full fine-tuning [29], while LoRA-FA introduces memory-efficient reparameterizations for adapter training [28].

Structural variations of LoRA have also been proposed. Delta-LoRA fine-tunes high-rank parameters through deltas of low-rank matrices [33], DoRA decomposes weights into magnitude and direction components for finer control [34], and VeRA employs vector-based random matrix adaptation [35]. LoHa introduces a low-rank Hadamard product formulation, while LoKr leverages a low-rank Kronecker product decomposition to enhance parameter efficiency and structural expressiveness [37,38]. Other refinements include LoRA+, which improves initialization and optimization [31], and X-LoRA, which adopts a mixture-of-experts design for LoRA modules and extends applications beyond language modeling to domains such as protein mechanics and molecular modeling [32]. These approaches modify the internal parameterization or optimization of LoRA adapters.

Recent extensions of LoRA have explored the dynamic allocation and structural refinement of low-rank adapters. Liao et al. propose Dynamic LoRA [14], which adaptively reallocates adapter capacity across layers during fine-tuning by estimating task-specific layer importance and incorporating input feature statistics, thereby moving beyond uniform adapter placement while retaining the core low-rank formulation. In contrast, Li et al. introduce Nested Low-Rank Adaptation (NoRA) [36], which restructures LoRA itself through a nested design with activation-aware SVD (AwSVD) initialization and a frozen outer adapter coupled with a trainable inner adapter, improving parameter efficiency and convergence. While both approaches enhance LoRA's efficiency, Dynamic LoRA focuses on adaptive allocation across layers using task sensitivity, whereas NoRA refines the internal parameterization and initialization of the adapters, highlighting complementary directions in the evolving PEFT design space.

2.2. Layer Specialization Substructures

Prior work has demonstrated that Transformer models exhibit a structured, non-uniform distribution of information across layers. Tenney et al. [10] showed that pretrained

BERT encodes linguistic phenomena in a hierarchical manner, where lower layers capture syntactic features while higher layers encode more abstract semantic information. This layer-wise specialization suggests that different layers contribute distinct functional roles rather than acting uniformly. Such findings imply that representational magnitude and structure are not randomly distributed but are organized during pretraining.

Beyond layer specialization, multiple studies have shown that attention heads are not equally important. Michel et al. [41] demonstrated that many attention heads can be pruned with a negligible impact on downstream performance, indicating significant redundancy within multi-head attention. Similarly, Voita et al. [12] identified that only a subset of heads perform "heavy lifting," while others contribute minimally. These results highlight that functional importance within Transformers is concentrated in specific components rather than being uniformly spread across all heads.

Further attribution-based analyses reinforce this perspective. Hao et al. [42] proposed methods to quantify head-level information interactions, revealing hierarchical contribution patterns within attention mechanisms. Liu et al. [43] examined structural differences between single-head and multi-head configurations, emphasizing representational diversity across heads. In decoder-focused analyses, Olsson et al. [48] identified induction heads that consistently implement specific algorithmic behaviors, demonstrating stable, functionally specialized substructures in autoregressive models. Collectively, these findings support the view that pretrained Transformers develop structured and layer-dependent representations, where certain layers and heads play disproportionately important roles. This body of evidence motivates selective adaptation strategies that prioritize structurally significant components rather than uniformly adapting all layers.

2.3. Layer Importance Metrics

Importance metrics have long been studied in the neural network literature. Early work by LeCun et al. on Optimal Brain Damage [44] and Hassibi et al. on Optimal Brain Surgeon [45] introduced Hessian-based importance measures for parameter pruning. These methods leverage the second-order information of the loss landscape, where parameters associated with higher curvature are considered more influential, as small perturbations can induce large changes in loss. In practice, importance is estimated using diagonal approximations of the Hessian to quantify parameter saliency. While theoretically well grounded, computing Hessian-based importance remains prohibitively expensive for large Transformer models.

Fisher information measures the expected sensitivity of the model output to parameter perturbations and has been applied to neural network pruning by Theis et al. [46] and widely used in continual learning frameworks. Fisher-based importance scores are derived from gradients of the log-likelihood with respect to model parameters and offer a strong theoretical grounding in information theory. In practice, however, estimating Fisher information requires computing expectations over gradients across data samples or batches, often involving blockwise approximations and additional aggregation. This makes Fisher-based methods computationally expensive for large Transformer models and motivates the exploration of lighter-weight importance metrics.

Tracking the magnitude of parameter updates provides another perspective on importance. This idea dates back to transfer learning methods such as Elastic Weight Consolidation [39], where the magnitude of weight changes (weight delta) reflects how strongly parameters adapt to new data. Similarly, gradient magnitudes indicate how much a parameter or layer influences the current loss and have been used for importance estimation in pruning, for example, through gradient norms [46]. Because raw gradients are inherently noisy and sensitive to optimization dynamics, methods such as AdaLoRA [13] apply

exponential smoothing to combine gradient and weight magnitudes into a more stable sensitivity score. Dynamic LoRA [14] used task-aware loss-based sensitivity to determine layer importance to dynamically assign varying ranks during training.

Finally, activation-based importance was popularized in neuron-pruning research as a simple signal of representational strength. Li et al. [47] introduced filter pruning by activation statistics, using the mean activation magnitude to remove uninformative filters. More recently AG-LoRA [10] demonstrated that activation-guided signals can also serve as reliable indicators for low-rank parameter adaptation in large language models. Their activation-guided Low-Rank Adaptation method showed that activation energy correlates closely with layer sensitivity. Most recently, ref. [49] used three metrics for determining layer contribution, namely projected residual norm (ResNorm), activation energy and layer coupling. They systematically showed that layer optimization is expensive for layers where the activations showcase weak input-dependent variation. NoRA [36] used activation statistics for informed initialization of the outer LoRA adapters.

3. Theoretical Analysis

3.1. The Classic LoRA Formulation

Consider a linear layer with frozen base weight $W_{\text{base}} \in \mathbb{R}^{k\times d}$. LoRA introduces a low-rank update as follows:

$$W' = W_{\text{base}} + \Delta W, \qquad \Delta W = BA, \tag{1}$$

where $A \in \mathbb{R}^{r\times d}$, $B \in \mathbb{R}^{k\times r}$, and $r \ll \min(d,k)$.

Given an input activation vector $x \in \mathbb{R}^d$, the layer output becomes

$$h(x) = W'x = W_{\text{base}}x + BAx. \tag{2}$$

The functional change induced by LoRA is therefore

$$\boxed{\Delta h(x) = BAx.} \tag{3}$$

3.2. Scaling Behavior with Respect to Activation Magnitude

If the input is scaled by a scalar α, we obtain

$$\Delta h(\alpha x) = BA(\alpha x) = \alpha \Delta h(x). \tag{4}$$

Thus, the LoRA-induced perturbation scales linearly with the magnitude of the activation. For any nonzero vector $x \in \mathbb{R}^d$, we can decompose it as

$$x = \|x\|_2 u, \qquad u = \frac{x}{\|x\|_2}, \qquad \|u\|_2 = 1, \tag{5}$$

where u is a *unit vector* (i.e., a vector with ℓ_2 norm equal to 1) that represents the direction of x.

Substituting $x = \|x\|_2 u$ into Equation (3), we obtain

$$\Delta h(x) = BA\big(\|x\|_2 u\big) = \|x\|_2\, BAu. \tag{6}$$

Taking the ℓ_2 norm,

$$\boxed{\|\Delta h(x)\|_2 = \|x\|_2 \|BAu\|_2.} \tag{7}$$

3.3. Formal Upper Bound on LoRA-Induced Perturbation

The induced (spectral) matrix norm of a matrix $M \in \mathbb{R}^{m\times n}$ is defined as

$$\|M\|_2 = \max_{\|v\|_2=1} \|Mv\|_2. \tag{8}$$

By definition of the maximum, for any unit vector v satisfying $\|v\|_2 = 1$, we have

$$\|Mv\|_2 \leq \|M\|_2. \tag{9}$$

Substituting $M = BA$ and $v = u$, we obtain

$$\|BA\|_2 = \max_{\|u\|_2=1} \|BAu\|_2. \tag{10}$$

$$\|BAu\|_2 \leq \|BA\|_2. \tag{11}$$

Combining with Equation (7),

$$\boxed{\|\Delta h(x)\|_2 \leq \|BA\|_2 \|x\|_2.} \tag{12}$$

Proposition 1. *For fixed LoRA parameters A and B, the maximum representational change induced by LoRA at a layer is upper bounded by the product of the spectral norm of BA and the activation norm $\|x\|_2$.*

3.4. Activation-Dependent Gradient Dynamics

Let $\mathcal{L}$ denote the training loss. From Equation (3), the LoRA contribution to the layer output is

$$\Delta h(x) = BAx. \tag{13}$$

Since A and B influence the loss only through Δh, we apply the chain rule. Define the upstream gradient δ as

$$\boxed{\delta = \frac{\partial \mathcal{L}}{\partial h}.} \tag{14}$$

Let z be defined as

$$z = Ax. \tag{15}$$

Then $\Delta h = Bz$.

3.4.1. Gradient w.r.t. B

Because $\Delta h = Bz$ is linear in B, the differential is

$$d(\Delta h) = (dB)\, z. \tag{16}$$

Using $d\mathcal{L} = \delta^\top dh = \delta^\top d(\Delta h)$,

$$d\mathcal{L} = \delta^\top (dB)\, z = \langle \delta z^\top,\, dB \rangle, \tag{17}$$

which yields

$$\boxed{\frac{\partial \mathcal{L}}{\partial B} = \delta z^\top = \delta (Ax)^\top.} \tag{18}$$

where $\langle X, Y \rangle$ denotes the Frobenius inner product (dot product for matrices) between two matrices X and Y.

3.4.2. Gradient w.r.t. A

Since $z = Ax$, we have $dz = (dA)\,x$, and thus

$$d(\Delta h) = B(dA)\,x. \tag{19}$$

Therefore,

$$d\mathcal{L} = \delta^\top B(dA)\,x = \langle B^\top \delta\, x^\top,\, dA \rangle, \tag{20}$$

which gives

$$\boxed{\frac{\partial \mathcal{L}}{\partial A} = B^\top \delta\, x^\top.} \tag{21}$$

We use the following identity for the Frobenius norm of an outer product. For vectors $u \in \mathbb{R}^m$ and $v \in \mathbb{R}^n$,

$$\|uv^\top\|_F = \|u\|_2\, \|v\|_2. \tag{22}$$

Taking Frobenius norms for Equation (18) and (21),

$$\boxed{\left\| \frac{\partial \mathcal{L}}{\partial A} \right\|_F \leq \|B^\top \delta\|_2 \|x\|_2,} \tag{23}$$

$$\boxed{\left\| \frac{\partial \mathcal{L}}{\partial B} \right\|_F \leq \|\delta\|_2 \|Ax\|_2.} \tag{24}$$

Thus, the gradient magnitude of both A and B scales proportionally with the activation magnitude.

Proposition 2. *The optimization dynamics of LoRA are multiplicatively modulated by the input activation norm $\|x\|_2$. Layers with small activation magnitudes induce proportionally smaller gradient updates in the LoRA parameters.*

3.5. Implications for Layer Selection

The above analysis shows that the activation norm $\|x\|_2$ appears multiplicatively in both the forward perturbation and the backward gradients of LoRA.

First, the change induced in the layer output satisfies

$$\|\Delta h(x)\|_2 \leq \|BA\|_2 \|x\|_2,$$

which means that, for fixed LoRA parameters, the maximum change that can be introduced at a layer scales with the activation magnitude.

Second, the gradient magnitudes of A and B also scale proportionally with $\|x\|_2$, implying that layers with smaller activations receive proportionally weaker parameter updates during training.

Therefore, layers with larger activation norms allow both stronger representational modification and stronger gradient flow under the same adaptation capacity. This motivates applying LoRA preferentially to layers where the activation norm $\|x\|_2$ is large, as such layers permit proportionally greater functional impact per parameter.

4. Proposed Methodology

Figure 1 depicts the overall flow of the proposed algorithm, Figure 2 shows the probe placement sites for collecting activation norms and the Figure 3 shows a comparison between standard LoRA implementation and our proposed method.

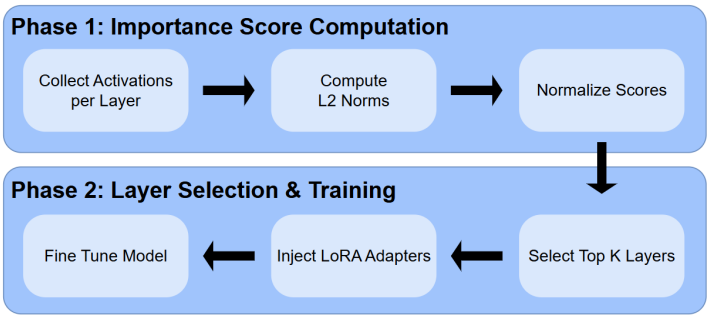


Figure 1. Act-LoRA: activation-guided layer selection for Low-Rank Adaptation.

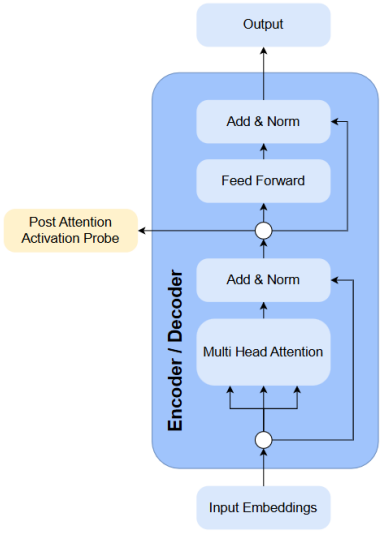


Figure 2. Probe placement in encoder or decoder blocks for collecting activation norms.

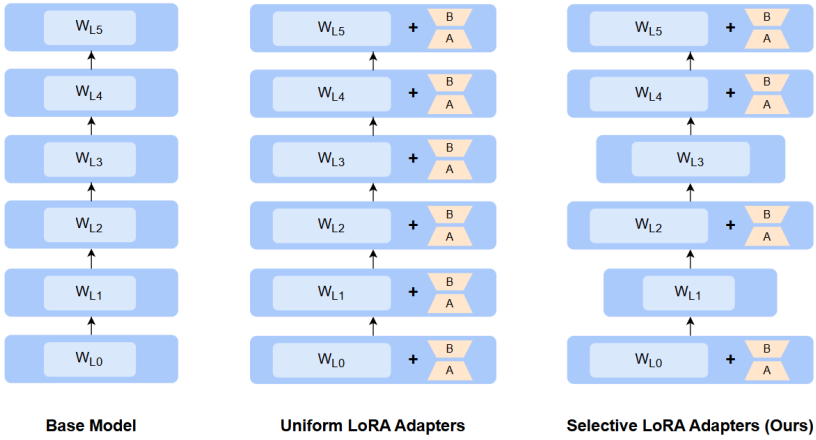


Figure 3. Selective LoRA schematic diagram. (**Left**): Base model, (**Center**): LoRA adapters applied uniformly to all layers, (**Right**): LoRA adapters applied to selected layers guided by activation norms.

4.1. Layer Selection Algorithm

Algorithm 1 and Figure 1 describe our Act-LoRA (activation-guided layer selection for LoRA) procedure, which selects a small set of relevant layers for LoRA insertion using

layer-wise activation statistics. The Algorithm has two distinct phases. During the first phase (probing phase), we collect the activations from each attention block, compute L2 norms per layer and normalize the score. In the second phase we select the top-k layers with the highest importance score and inject LoRA adapters to these layers, and finally train the selected layers with the frozen base model (Layer Selection and Training).

Algorithm 1 Act-LoRA: Activation-Guided Layer Selection for LoRA

procedure ACT-LORA($\mathcal{M}, \mathcal{L}, \mathcal{X}, S, K, r, E, \eta$)
  Freeze $\mathcal{M}$ with base LoRA adapters
  Register forward hooks on all candidate layers $\ell \in \mathcal{L}$
  for $t = 1$ to S **do**
    Sample mini-batch $x_t \sim \mathcal{X}$
    $y_t \leftarrow \mathcal{M}(x_t)$
    for each layer $\ell \in \mathcal{L}$ **do**
      $A_\ell \leftarrow A_\ell + \frac{1}{T(x_t)} \sum_{i=1}^{T(x_t)} \|h_\ell(x_t)_i\|_2$
    end for
  end for
  $A_\ell \leftarrow A_\ell / S$
  $s_\ell \leftarrow \dfrac{A_\ell - \min_j A_j}{\max_j A_j - \min_j A_j}$
  Sort layers by descending s_ℓ
  $\mathcal{L}_{\text{sel}} \leftarrow$ top-K layers
  for each layer $\ell \in \mathcal{L}$ **do**
    if $\ell \in \mathcal{L}_{\text{sel}}$ **then**
      Insert LoRA adapter of rank r into layer ℓ
    else
      Freeze layer ℓ
    end if
  end for
  Train $\mathcal{M}$ for E epochs on $\mathcal{X}$ using AdamW with learning rate η
  return fine-tuned model $\mathcal{M}$
end procedure

4.1.1. Why L2 Norm?

The L2 norm provides an energy-based measure that captures the aggregate signal strength contributed by a layer across tokens and batches. In contrast, mean activation values can suffer from sign cancellation, causing highly active but balanced layers to appear unimportant, while max-based metrics are dominated by rare outliers and are sensitive to batch composition. By aggregating contributions across all activation dimensions, the L2 norm yields a stable and comparable importance estimate across layers, making it well suited for guiding selective adaptation decisions under a fixed parameter budget.

4.1.2. Algorithm

We denote the underlying pretrained model as $\mathcal{M}$ and the set of candidate layers as $\mathcal{L}$. The downstream task dataset is represented by $\mathcal{X}$. The algorithm begins with a probing phase, controlled by the hyperparameter S, which specifies the number of mini batches used for activation collection. During this phase, $\mathcal{M}$ is kept frozen and lightweight forward hooks are registered on each attention block (post-attention, Figure 2) $\ell \in \mathcal{L}$ to extract their activations $h_\ell(x)$ for each batch x.

For each probing batch x_t, the model performs a forward pass, producing activations $h_\ell(x_t) \in \mathbb{R}^{T(x_t) \times d_\ell}$ for every layer ℓ, where $T(x_t)$ denotes the batch size (in tokens or

sequences) and d_ℓ is the layer's hidden dimension. We compute the per-batch activation magnitude via the mean ℓ_2 norm,

$$\frac{1}{T(x_t)} \sum_{i=1}^{T(x_t)} \|h_\ell(x_t)_i\|_2,$$

and accumulate these values into an activation score A_ℓ for each layer. After S probing batches, we obtain the mean activation level

$$A_\ell \leftarrow \frac{A_\ell}{|D_{\text{probe}}|},$$

where D_{probe} denotes the set of all probing batches.

Since raw activation magnitudes vary across layers, we normalize them to a common scale. Specifically, the normalized activation score

$$s_\ell = \frac{A_\ell - \min_j A_j}{\max_j A_j - \min_j A_j}$$

maps every layer's activation into the interval $[0, 1]$, with larger values indicating higher layer-task alignment.

Next, we rank all layers by s_ℓ and select the top-K layers, denoted $\mathcal{L}_{\text{sel}}$, which are interpreted as the most influential layers. We then insert LoRA adapters of rank r exclusively into these layers, while freezing all others. This design allows the model to allocate adaptation capacity only to the layers that exhibit the strongest representational signal.

Finally, the model is fine-tuned for E epochs using AdamW with learning rate η, updating only the LoRA parameters.

4.2. Time Complexity: Phase 1 (Importance Score Computation)

Let $L = |\mathcal{L}|$ denote the number of candidate layers, S the number of probe steps, B the batch size, T the sequence length, and d the hidden dimension.

The cost for a single probe step corresponds to a forward pass through the frozen base model as follows:

$$O(\text{Fwd}(B, T)), \tag{25}$$

where $\text{Fwd}(B, T)$ is a function of the batch size B and sequence length T. Accordingly, the cost for S probe steps is

$$O(S \cdot \text{Fwd}(B, T)). \tag{26}$$

In addition to the forward pass, the probe phase computes activation norm statistics across L candidate layers. The cost of activation norm reduction is

$$O(LBTd), \tag{27}$$

where B is the batch size, T the sequence length, and d the hidden dimension. Over S probe steps, the total cost of activation norm reduction becomes

$$O(S \cdot LBTd). \tag{28}$$

After collecting activation statistics, the candidate layers are ranked by their importance scores. This ranking and selection step requires sorting L layers, incurring a cost of

$$O(L \log L). \tag{29}$$

Combining all components, the total time complexity of the probe phase is

$$O(S(\text{Fwd}(B,T) + LBTd) + L\log L). \tag{30}$$

Under standard dense self-attention, the forward pass includes a quadratic term in the sequence length, i.e., $\text{Fwd}(B,T) = \Omega(BT^2d)$ (often $\Omega(DBT^2d)$ for D layers). Consequently, for typical Transformer configurations where $T \gg L$, the forward-pass cost dominates both the activation norm reduction and the sorting overhead:

$$O(\text{Fwd}(B,T)) \gg O(LBTd), \quad O(\text{Fwd}(B,T)) \gg O(L\log L).$$

Hence, the time complexity of layer importance scoring is dominated by the forward pass and can be approximated as

$$T(\text{Layer Importance Scoring}) \approx O(S \cdot \text{Fwd}(B,T)). \tag{31}$$

4.3. Space Complexity: Phase 1 (Importance Score Computation)

The selection phase maintains a scalar activation statistic per candidate layer and corresponding scores, requiring $O(L)$ additional memory. The probe phase is executed in inference mode, so no backward graph is retained. Thus, the peak memory footprint is dominated by the base model's forward-pass memory for a single batch:

$$O(\text{ForwardMemory}(B,T) + L).$$

4.4. Time Complexity: Phase 2 (Layer Selection and LoRA Training)

Consider a Transformer model with L layers and hidden dimension d. Let a mini batch contain T token positions in total (i.e., batch size times sequence length). Low-Rank Adaptation (LoRA) replaces a linear projection $W \in \mathbb{R}^{d\times d}$ with

$$W + \Delta W, \quad \text{where } \Delta W = BA,$$

and $A \in \mathbb{R}^{r\times d}$, $B \in \mathbb{R}^{d\times r}$ with $r \ll d$.

4.4.1. LoRA Computation per Projection

Given hidden states $X \in \mathbb{R}^{T\times d}$, the additional forward computation introduced by LoRA is

$$X\Delta W = XA^\top B^\top.$$

This is evaluated as two matrix multiplications:

$$U = XA^\top, \qquad U \in \mathbb{R}^{T\times r}, \tag{32}$$

$$X\Delta W = UB^\top, \qquad X\Delta W \in \mathbb{R}^{T\times d}. \tag{33}$$

Each multiplication costs $\widetilde{O}(Tdr)$, yielding an additional forward-pass cost of

$$\widetilde{O}(2Tdr)$$

per LoRA-adapted projection.

During training, gradients must be computed with respect to A and B, introducing backward-pass matrix multiplications of the same asymptotic complexity. Hence, the total LoRA-induced training cost per projection is

$$\widetilde{O}(Tdr) \tag{34}$$

Suppose LoRA is applied to m linear projections per Transformer layer (e.g., $m = 2$ for query and value projections). The additional training cost per adapted layer is therefore

$$\widetilde{O}(mTdr) \tag{35}$$

If LoRA is applied to only k out of the L layers, the total LoRA-specific training cost per mini batch becomes

$$\widetilde{O}(mkTdr) \tag{36}$$

4.4.2. Scaling with the Number of Adapted Layers

For fixed T, d, and r, the LoRA-induced compute scales linearly with the number of adapted layers. Relative to adapting all L layers, restricting LoRA to k layers reduces the LoRA-specific training cost by a factor of k/L.

4.5. Space Complexity: Phase 2 (Layer Selection and Training)

Each layer adapted with Q+V LoRA introduces

$$P_{\text{layer}} = 4dr. \tag{37}$$

parameters (two projections, each with $A \in \mathbb{R}^{r\times d}$ and $B \in \mathbb{R}^{d\times r}$).

Adapting all L layers results in

$$P_{\text{all}} = 4drL, \tag{38}$$

while adapting only the top-k layers yields

$$P_{\text{top-}k} = 4drk. \tag{39}$$

Thus, the number of trainable parameters also scales linearly with k, providing proportional savings in both parameter storage and gradient/optimizer memory.

5. Experimental Setup

5.1. Datasets

We evaluate our proposed approach on the General Language Understanding Evaluation (GLUE) benchmark [50], a widely used suite of natural language understanding tasks. GLUE consists of diverse datasets that assess different aspects of linguistic reasoning, including single-sentence classification, sentence pair similarity, and natural language inference. Using the full GLUE benchmark ensures that our evaluation is consistent with prior work on LoRA and its variants.

5.2. Models

To evaluate the generality of our approach, we conduct experiments on both encoder-only and decoder-only Transformer architectures. DeBERTaV3-Base (184 M parameters) [51] is trained on a single NVIDIA L4 GPU without additional memory optimizations. In contrast, experiments on LLaMA-3.1-8B (8B parameters) [11] are performed using Pytorch's implementation of Distributed Data Parallel (DDP) training across four NVIDIA A100 GPUs. Base model weights are frozen and quantized to 4-bit NF4 with double quantization, all forward/backward computation is performed in bfloat16, and bf16 LoRA adapters (applied to Q/V projections) are trained using AdamW under PyTorch (Version 2.1.x), SDPA attention. No quantization or distributed training is used for DeBERTaV3-Base. For experiments including BERT-Base, we used 1× A100 40 GB RAM.

5.3. Low-Rank Adapter Module Selection

Before selecting which modules to adapt, it is important to note that LoRA can be applied to any subset of a Transformer's linear projections—most commonly the attention matrices W_q, W_k, W_v and the two feed-forward layers (FFN_1, FFN_2). These choices introduce different computational and parameter costs. For our paper, we have chosen Query (W_q) and Value (W_v) projections for adaptation, as LoRA adapters applied to W_k provide minimal gains and FFN_1, FFN_2 are expensive—refer to [6] for details.

For inference evaluation, we kept the LoRA adapter modules unmerged. The reason for unmerged adapters is that swapping LoRA adapters during inference mode is expensive, and one of the limitations of LoRA [6], hence measuring the compute savings in this unmerged mode is more meaningful.

5.4. Training and Testing Hyperparameters

We keep the benchmarking criteria simple: each GLUE dataset is fine-tuned for 3 epochs, with a learning rate of 2×10^{-4}, an input sequence length of 128 tokens, a training batch size of 32, an inference batch size of 64, FP16 precision, and the AdamW optimizer. The base model is probed for 200 optimizer steps in DeBERTaV3-Base and 10 steps for Llama3.1-8B for collecting activation norms.

We compare Act-LoRA against standard LoRA (rank = 8, lora_alpha = 16) and AdaLoRA (init_rank = 12, target_rank = 8). Each experiment is run with 3 seeds, and we report the mean and standard deviation for all metrics.

For LoRA and Act-LoRA (ours) we have kept the hyperparameters the same, and they do not defer based on task.

The original AdaLoRA benchmark employs dataset-specific hyperparameters, extended training schedules, and tuned values for t_{init}, t_{final}, and Δt. For a fair comparison, we rescale t_{init}, t_{final}, and Δt to match our 3-epoch training budget while keeping all other hyperparameters at their default settings. Although AdaLoRA dynamically reallocates rank during training, the physical adapter dimensions remain fixed; therefore, we report the total trainable parameter count as implemented.

5.5. Evaluation Metrics

To assess the trade-offs between accuracy and efficiency across different parameter-efficient fine-tuning methods, we report five complementary metrics. These capture both downstream task quality and system-level performance under realistic training and inference conditions. All measurements are computed relative to the LoRA baseline for each model family. Refer to Table 2 for model evaluation metric details.

Table 2. Evaluation metrics used to compare accuracy retention, training efficiency, inference performance, and memory footprint. All metrics are reported relative to the LoRA baseline within each model family.

Metric	Type	Formula	Description
GLUE Score	Task Quality	GLUE	Unweighted mean of task-specific GLUE metrics.
GLUE Δ (%)	Task Quality	$100 \times \frac{\text{GLUE}_{\text{method}} - \text{GLUE}_{\text{LoRA}}}{\text{GLUE}_{\text{LoRA}}}$	Relative change in downstream task performance with respect to LoRA.
Trainable Parameters (%↓)	Training Time	$100 \times \frac{P_{\text{LoRA}} - P_{\text{method}}}{P_{\text{LoRA}}}$	Reduction in the number of trainable parameters.
GPU Hours (%↓)	Training Time	$100 \times \frac{\text{GPUh}_{\text{LoRA}} - \text{GPUh}_{\text{method}}}{\text{GPUh}_{\text{LoRA}}}$	Percentage reduction in total training compute.
Peak Memory (%↓)	Training Time	$100 \times \frac{\text{Mem}_{\text{LoRA}} - \text{Mem}_{\text{method}}}{\text{Mem}_{\text{LoRA}}}$	Percentage reduction in peak GPU memory usage during training and inference.
Latency (%↓)	Inference Time	$100 \times \frac{\text{Lat}_{\text{LoRA}} - \text{Lat}_{\text{method}}}{\text{Lat}_{\text{LoRA}}}$	Percentage reduction in mean per-example inference latency (ms).
Throughput (%↑)	Inference Time	$100 \times \frac{\text{TP}_{\text{method}} - \text{TP}_{\text{LoRA}}}{\text{TP}_{\text{LoRA}}}$	Percentage increase in inference throughput (samples/s).

It is important to note that prior LoRA variants such as AdaLoRA, DyLoRA, and AutoLoRA do not consistently report the full spectrum of performance metrics considered in this work.

6. Exprimental Results

6.1. Comparison with LoRA Methods

6.1.1. GLUE Score

Tables 3 and 4 show that Act-LoRA (our method) consistently preserves most of the baseline LoRA performance while using substantially fewer trainable parameters across both backbones. On DeBERTa-V3-Base, Act-LoRA with k = 6 achieves near parity with LoRA while reducing the parameter count by roughly 50%, with only marginal drops on MNLI, QNLI, SST-2, and STS-B, and even the more aggressive k = 2 setting retains a strong overall accuracy despite using over 80% fewer parameters. A similar pattern is observed for LLaMA-3.1-8B, where Act-LoRA with k = 24 reduces trainable parameters by approximately 26% while matching the LoRA performance, and the k = 16 variant cuts the parameter budget by nearly 50% with only a modest accuracy loss, particularly preserving the performance on high-signal tasks such as MNLI, QNLI, and SST-2.

Table 3. GLUE performance across all datasets for different LoRA variants on DeBERTa-V3-Base and LLaMA-3.1-8B. We report Matthews' correlation coefficient for CoLA, Pearson's correlation for STS-B, and accuracy for all other tasks. LoRA (r = 8), ActLoRA (r = 8), AdaLoRA (target_r = 8, init_r = 12).

Model	Method	# Params	MNLI	QNLI	QQP	SST-2	MRPC	RTE	CoLA	STS-B	WNLI
DeBERTa-V3-Base	LoRA	300 K	$90.0_{\pm 0.1}$	$93.6_{\pm 0.2}$	$90.1_{\pm 0.0}$	$95.0_{\pm 0.2}$	$85.8_{\pm 1.1}$	$66.9_{\pm 3.0}$	$62.9_{\pm 0.9}$	$85.0_{\pm 1.0}$	$54.5_{\pm 3.2}$
	AdaLoRA	445 K	$89.4_{\pm 0.1}$	$92.3_{\pm 0.1}$	$89.0_{\pm 0.0}$	$93.3_{\pm 0.2}$	$81.2_{\pm 0.0}$	$51.5_{\pm 4.1}$	$0.0_{\pm 0.0}$	$57.2_{\pm 0.2}$	$52.1_{\pm 7.3}$
	Act-LoRA ($k = 2$)	50 K	$87.6_{\pm 0.4}$	$91.4_{\pm 0.2}$	$88.2_{\pm 0.2}$	$94.9_{\pm 0.2}$	$75.9_{\pm 0.5}$	$55.5_{\pm 2.5}$	$58.4_{\pm 1.1}$	$81.9_{\pm 1.5}$	$56.3_{\pm 0.0}$
	Act-LoRA ($k = 6$)	150 K	$89.8_{\pm 0.2}$	$93.4_{\pm 0.0}$	$89.4_{\pm 0.1}$	$95.5_{\pm 0.4}$	$78.0_{\pm 0.4}$	$61.1_{\pm 4.1}$	$61.4_{\pm 1.3}$	$84.3_{\pm 1.5}$	$56.3_{\pm 0.0}$
LLaMA-3.1-8B	LoRA	3.4 M	$91.2_{\pm 0.2}$	$95.4_{\pm 0.2}$	$91.7_{\pm 0.1}$	$96.7_{\pm 0.3}$	$78.9_{\pm 3.2}$	$60.5_{\pm 4.0}$	$66.2_{\pm 0.7}$	$70.3_{\pm 6.1}$	$47.4_{\pm 3.5}$
	AdaLoRA	5.1 M	$91.3_{\pm 0.1}$	$95.0_{\pm 0.2}$	$90.1_{\pm 0.0}$	$95.9_{\pm 0.2}$	$66.8_{\pm 1.7}$	$55.5_{\pm 2.7}$	$36.2_{\pm 2.7}$	$28.7_{\pm 7.2}$	$52.6_{\pm 9.9}$
	Act-LoRA ($k = 16$)	1.7 M	$89.5_{\pm 0.1}$	$93.9_{\pm 0.1}$	$90.7_{\pm 0.0}$	$96.3_{\pm 0.1}$	$71.1_{\pm 1.7}$	$57.9_{\pm 3.3}$	$63.9_{\pm 0.8}$	$61.9_{\pm 6.5}$	$48.4_{\pm 4.3}$
	Act-LoRA ($k = 24$)	2.5 M	$91.0_{\pm 0.1}$	$95.3_{\pm 0.2}$	$91.2_{\pm 0.1}$	$96.5_{\pm 0.2}$	$75.2_{\pm 3.2}$	$60.6_{\pm 5.7}$	$63.8_{\pm 0.9}$	$70.1_{\pm 5.5}$	$52.1_{\pm 7.0}$

Table 4. Aggregate training and inference efficiency comparison across LLaMA-3.1-8B and DeBERTaV3-Base variants. Training GPUh (GPUh), Training GPU Memory Allocation (GiB) (Mem), Time to First Token (ms) (TTFT), Trainable Params Count (Params), Samples Per Sec (Throughput), Inference Latency (ms) (Latency). LoRA (r = 8), ActLoRA (r = 8), AdaLoRA (target_r = 8, init_r = 12).

Model	Method	GLUE	GPUh	Mem	TTFT	Params	Throughput	Latency
LLaMA-3.1-8B	Lora	77.59	3.100	15.98	100.23	3,416,576	10.21	748.28
	Adalora	68.32	3.082	51.47	105.12	5,121,280	9.55	737.68
	ActLoRA (k = 16)	78.61	1.918	15.83	98.75	1,712,128	10.47	701.41
	ActLoRA (k = 24)	78.96	3.036	16.40	98.95	2,564,608	10.18	739.76
DeBERTaV3-Base	Lora	80.42	6.657	4.87	35.84	296,450	34.84	28.02
	Adalora	70.07	8.725	4.59	46.96	444,194	32.81	29.86
	ActLoRA (k = 2)	77.09	3.999	4.74	26.26	50,690	41.53	23.40
	ActLoRA (k = 6)	80.21	5.210	4.78	28.18	148,994	38.62	25.21

6.1.2. Training Hours (GPUh)

In Tables 4 and 5, across both backbones and task scales, the GPU hour trends reveal a consistent efficiency advantage for Act-LoRA relative to the LoRA baseline and, in most cases, AdaLoRA. On DeBERTaV3-Base trained on a single L4 GPU, Act-LoRA substantially

reduces GPU hours across both large and small GLUE tasks, with savings increasing as the number of selected layers decreases. In particular, Act-LoRA with $k = 6$ achieves an approximately 20–25% GPU hour reduction on large tasks, while the more aggressive $k = 2$ setting yields up to 40% savings, as summarized in Table 5.

A similar but attenuated trend is observed for LLaMA-3.1-8B on four A100 GPUs, where overall GPU hour costs are dominated by model scale. Additionally, this model was trained in Distributed Data Parallel mode, hence the number of samples processed per GPU is reduced by a factor of four because we had four GPUs taining the same model parallely. Moreover DDP introduces its gradient synchronization overheads, which were not captured because it is out of scope for this study. Nevertheless, Act-LoRA continues to provide consistent reductions, with $k = 16$ and $k = 24$ reducing GPU hours by approximately 8% and 2%, respectively, relative to LoRA.

Table 5. Trade-off analysis comparing GLUE score, relative GLUE change (Δ GLUE %), GPU hour savings (GPUh %↓), latency reduction (Latency %↓), throughput (Throughput) improvement, peak memory reduction (Peak Mem %↓), and trainable parameter reduction relative to LoRA (Params %↓). DeB results correspond to DeBERTaV3-Base on a single L4 GPU; LL results correspond to LLaMA-3.1-8B on four A100 GPUs.

Model	Method	GLUE	ΔGLUE (%)	GPUh (%↓)	Latency (%↓)	Throughput (%↑)	Peak Mem (%↓)	Params (%↓)
DeB	LoRA ($r = 8$)	0.801	–	–	–	–	–	–
	AdaLoRA ($r = 8$)	0.670	−16.3	−31.0	−17.5	−5.8	+5.86	−49.7
	Act-LoRA ($k = 6$)	0.792	−1.1	+21.7	+0.7	+10.8	+1.97	+49.7
	Act-LoRA ($k = 2$)	0.772	−3.6	+40.0	+7.8	+19.2	+2.85	+82.9
LL	LoRA ($r = 8$)	0.780	–	–	–	–	–	–
	AdaLoRA ($r = 8$)	0.689	−11.7	+0.5	+1.1	−6.4	−213.3	−49.8
	Act-LoRA ($k = 24$)	0.779	−0.13	+2.1	+1.1	−0.38	−2.53	+24.9
	Act-LoRA ($k = 16$)	0.755	−3.2	+7.7	+5.0	+2.75	+0.28	+49.9

6.1.3. Training Memory Consumption (Peak Memory)

On DeBERTaV3-Base trained on a single L4 GPU, memory usage remains relatively stable across LoRA, AdaLoRA, and Act-LoRA variants, with only minor fluctuations. In contrast, on the much larger LLaMA-3.1-8B model trained on four A100 GPUs, clear differences emerge: AdaLoRA exhibits substantially higher memory allocation than the LoRA baseline. Act-LoRA, by selectively inserting adapters into a small subset of activation-salient layers, avoids this overhead and maintains memory usage close to or slightly above the LoRA baseline, even for higher-capacity settings such as $k = 24$.

6.1.4. Inference Latency

Table 5 reports the relative latency change of different fine-tuning methods with respect to the LoRA baseline across GLUE tasks. On DeBERTaV3-Base trained on a single L4 GPU, Act-LoRA consistently achieves substantial latency reductions across all tasks, with the more aggressive $k = 2$ configuration yielding the largest gains, often exceeding 15–20% relative to LoRA. Even the less aggressive $k = 6$ setting provides stable latency improvements of approximately 8–12%. On the other hand, AdaLoRA frequently exhibits negative latency improvements on DeBERTa. A similar trend is observed on LLaMA-3.1-8B, where Act-LoRA with $k = 16$ achieves consistent latency reductions of roughly 4–7% across most tasks, while the higher-capacity $k = 24$ variant offers smaller but still positive gains.

6.1.5. Throughput (Samples per Second)

Table 5 compares the relative throughput change of different fine-tuning methods with respect to the LoRA baseline across GLUE tasks. On DeBERTaV3-Base trained on a single L4 GPU, Act-LoRA consistently delivers substantial throughput improvements across all tasks, with gains typically in the 15–20% range for the more aggressive $k = 2$ configuration and around 8–12% for $k = 6$. A similar pattern is observed on LLaMA-3.1-8B, where Act-LoRA with $k = 16$ achieves modest but consistent throughput gains of approximately 2–6% across most tasks, while the higher-capacity $k = 24$ variant provides smaller improvements.

6.2. Comparison with State-of-the-Art PEFT Methods

We compared our ActLoRA with SOTA methods implemented under the Huggingface PEFT library. We trained the BERT-base (12 Layers) model for 1000 optimizer steps with FP16 precision on the SST-2 dataset. We kept the learning rate at 2×10^{-4} for ActLoRA (k = 6), LoRA, Full Fine-Tuning, LoKr and LoHA. IA3, VeRA and AdaLoRA used a learning rate of 5×10^{-4}.

Table 6 represents the aggregate accuracy, loss, GPUh and memory collected across three seeds. Figure 4 visualizes the GPUh breakdown and memory footprint during different training phases. Figure 5 visualizes the accuracy and loss vs. trainable parameters count across three seeds.

Table 6. Comparison of fine-tuning methods on SST-2 (1000 steps, 3 seeds). Accuracy (Acc) and Loss are reported as mean ± standard deviation in percentage format. GPU hours (GPUh) are derived from phase-level timing totals. Peak allocated memory (Mem) corresponds to maximum CUDA allocation (GiB) observed during training. Trainable parameters (Params) are reported as absolute count with percentage of total model parameters in parentheses. ↑: Higher is better, ↓: Lower is better.

Method	Acc (%↑)	Loss (%↓)	GPUh	Mem	Params (%)
Full FT	90.79 ± 0.26	22.90 ± 0.40	0.0134	3.73	109,483,778 (100.00)
IA3	85.40 ± 0.59	33.20 ± 1.00	0.0128	1.14	112,898 (0.10)
VeRA	84.63 ± 0.90	37.50 ± 2.00	0.0145	1.33	20,162 (0.02)
LoKr	80.77 ± 1.55	40.60 ± 2.00	0.0145	0.95	27,650 (0.03)
LoHA	85.74 ± 0.63	33.40 ± 1.00	0.0154	0.95	591,362 (0.54)
LoRA	88.99 ± 0.30	27.20 ± 1.00	0.0142	1.50	296,450 (0.27)
AdaLoRA	86.16 ± 0.46	32.20 ± 1.00	0.0209	1.58	444,194 (0.40)
ActLoRA (Ours)	88.38 ± 0.29	28.90 ± 1.00	0.0107	1.69	148,994 (0.14)

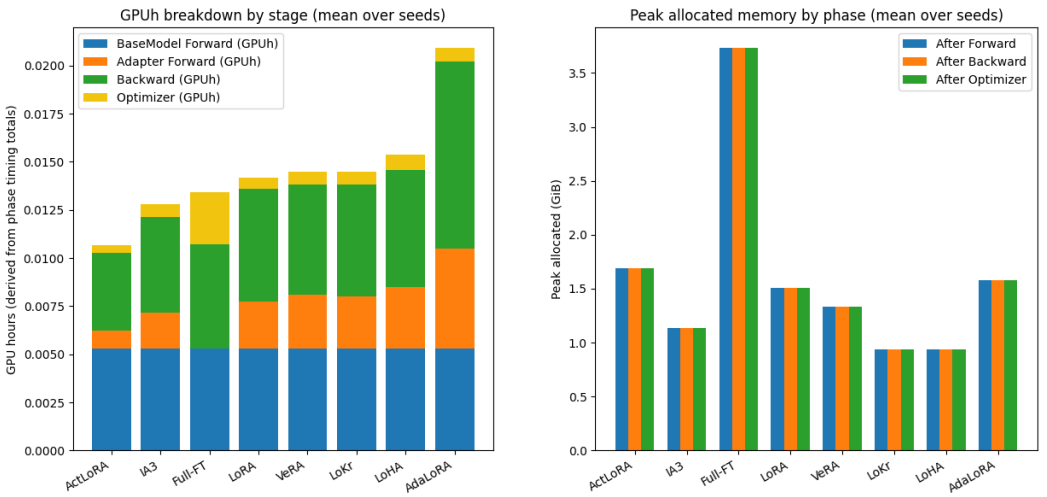


Figure 4. Phase-wise GPU time and memory comparison with SOTA methods. (**Left**): Per-step training time decomposed into forward, backward, and optimizer phases. (**Right**): GPU memory usage measured after forward, backward (peak), and optimizer phases.

In Figure 4, the GPUh breakdown shows that backward computation dominates the training cost across all methods, while the base forward cost remains largely constant. ActLoRA achieves the lowest overall GPUh among high-performing PEFT methods, demonstrating computational efficiency from selective adaptation. Memory usage remains substantially lower for PEFT approaches compared to full fine-tuning, with most methods operating around 1–1.7 GiB versus 3.7 GiB for FFT, highlighting the benefits of PEFT methods for reducing the memory footprint.

The log-scale visualization in Figure 5 reveals that parameter counts span several orders of magnitude, yet accuracy does not scale proportionally. LoRA and ActLoRA achieve the highest accuracy and lowest loss second to full fine-tuning while using three orders fewer trainable parameters. ActLoRA maintains a competitive performance at roughly half the LoRA parameter budget, indicating that selective layer adaptation preserves most of the useful update capacity. In contrast, aggressively compressed methods such as LoKr and VeRA show noticeable accuracy and loss degradation.

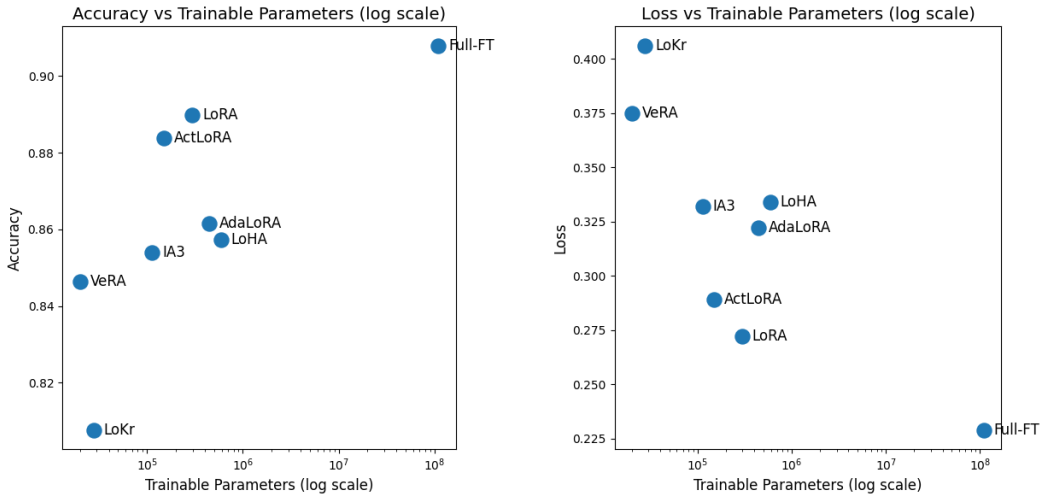


Figure 5. Accuracy and loss comparison with PEFT methods. (**Left**): Accuracy vs. trainable parameters count (log scale). (**Right**): Loss vs. trainable parameters count (log scale).

Compared to LoRA, the ActLoRA reduced compute by 25% while maintaining comparable accuracy, but memory remained similar. Compared to full fine-tuning, ActLoRA reduced compute by 20% and memory by 55%, with only a 2.4% absolute accuracy drop for BERT-Base on the SST-2 dataset.

6.3. Sweep Study

We conducted a controlled profiling study on BERT-Base (12 encoder layers) on the SST-2 dataset over three training epochs to measure how increasing the top-k layer selection impacts computational cost. Specifically, we recorded phase-wise GPU time (forward, backward, and optimizer) and memory consumption (after forward, backward peak, and after optimizer step) for $k \in \{2, 4, 6, 8, 10, 12\}$.

Figure 6 and Table 7 present the phase-wise GPU time and memory breakdown for top-k LoRA layer adaptation. As k increases from two to 12, total per-step training time rises from 33.74 ms to 51.23 ms (approximately +52%), with the backward pass contributing the largest share of the growth. In contrast, peak GPU memory increases modestly from 919.3 MiB to 1024.0 MiB (approximately +11%), consistently occurring after the backward phase.

Figure 7 shows the accuracy and loss trends achieved during the sweep study. We can observe that the accuracy is practically similar from k = 4 to k = 12, whereas the loss achieves its minimum at k = 6 and k = 8. Thus, from both the charts we can infer that k = 6 is the sweet spot for maximizing accuracy and loss for this setup.

The results show a clear, near-linear increase in both time and memory as k grows. The backward pass consistently dominates compute time, while the backward peak dominates memory usage, which is expected since gradient storage and intermediate activations scale with the number of adapted layers. The forward (base model + adapter) and optimizer costs increase more modestly but still track k proportionally. Importantly, no super-linear explosion is observed, indicating that selective top-k adaptation scales predictably and remains computationally manageable even as more layers are included. The relatively modest memory variation is expected, as LoRA adapter parameters constitute only a small fraction (approximately 1%) of the overall model parameters, meaning that the base model weights continue to dominate total GPU memory usage.

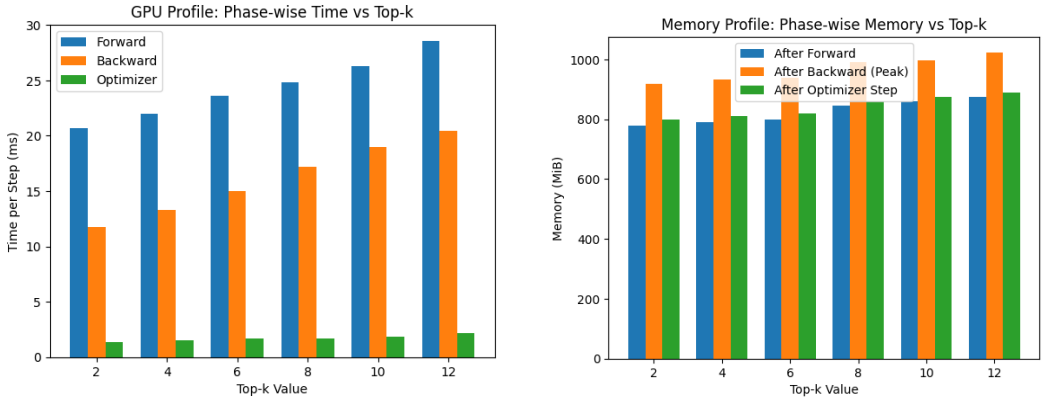


Figure 6. Phase-wise GPU time and memory scaling for top-*k* LoRA layer adaptation with activation norm-based importance score. (**Left**): Per-step training time decomposed into forward (base model + adapter), backward, and optimizer phases. (**Right**): GPU memory usage measured after forward, backward (peak), and optimizer phases.

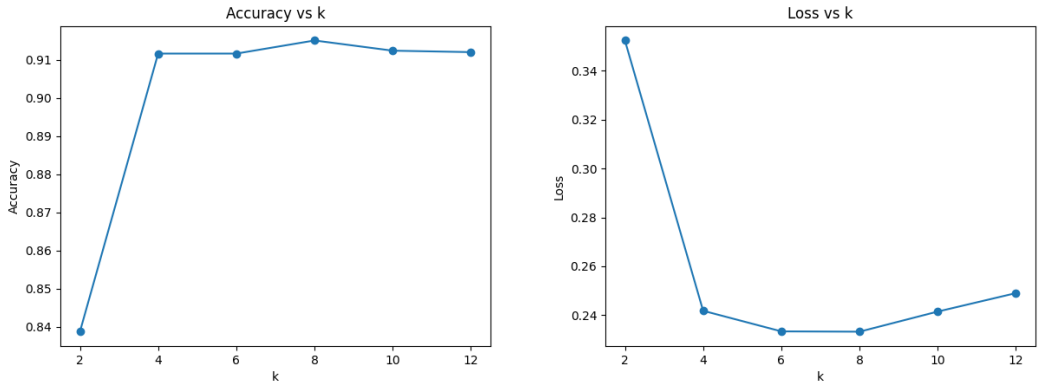


Figure 7. Accuracy and loss sweep study.

Table 7. Phase-wise GPU time and memory breakdown for top-*k* LoRA adaptation. Peak memory consistently occurs after the backward pass. Compute scales approximately linearly with *k*, while memory growth remains modest.

Top-*k*	GPU Time per Step (ms)				GPU Memory (MiB)		
	Forward	Backward	Optimizer	Total	After Forward	After Backward (Peak)	After Step
2	20.65	11.72	1.37	33.74	780	919.3	800
4	21.95	13.26	1.49	36.70	790	932.0	810
6	23.57	14.99	1.69	40.25	800	940.0	820
8	24.81	17.17	1.65	43.62	845	992.0	860
10	26.27	18.95	1.83	47.05	860	998.0	875
12	28.58	20.44	2.21	51.23	875	1024.0	890

7. Discussion

In this section we analyse the results and address the research questions. Section 7.1 discusses the observations from our benchmarking and its implications. Section 7.2 talks about the cumulative importance threshold mapping that allows us to identify the optimal starting point for selecting k. These sections address our RQ-1: Is selectively adapting a subset of layers sufficient to retain a strong task performance?

Next we perform empirical studies to understand the behavior of using activation norms as a layer selection signal. Section 7.5 compares the ablation results of activation norms with other layer importance metrics whereas Section 7.3 talks about the stable nature of activation norms against gradient norms. These sections address the RQ-2: Do activation norms provide a stable and informative signal for layer selection?

Finally Section 7.4 talks about our RQ-3: Is the activation norms-based importance score task-dependent?

7.1. Performance

Overall, we observe that adapting only the top-K layers preserves most of the downstream accuracy while reducing both training and inference cost. A second key observation is that the impact of top-k selection is strongly dataset-size-dependent. This is evidenced by the larger performance degradation on low-resource GLUE tasks (WNLI, RTE, CoLA, STS-B). Thus, under these training conditions, we can infer that top-K layer selection is best suited for medium- to high-resource datasets, where sufficient data allows the model to compensate for a reduced learning capacity.

We also observe unstable behavior for AdaLoRA under our experimental setup in Section 6.1, especially for low-resource datasets. We suspect two major reasons for this behavior. Firstly, AdaLoRA uses a smoothed gradient and weight product as the driving signal. The gradient and weight update is highly dependent on the optimizer and the learning rate choice. Moreover, gradients are highly noisy and may explode or vanish. Secondly, we experimented in a constrained three-epoch training budget due to limited compute resources. AdaLoRA relies on a multi-phase training schedule consisting of an initial over-parameterized phase, a structured pruning phase, and a post-pruning refinement phase, each governed by task-specific hyperparameters. Properly tuning these schedules requires longer training horizons and per-task calibration. Performing these per-task calibrations is out of scope for this study as it is a resource-intensive endeavour and we consider LoRA as our primary baseline. Perhaps a fixed max_step-based training schedule with early stopping would have helped in better convergence compared to a fixed three epochs schedule.

Section 6.2 attempts to study the SOTA fine-tuning technique on the SST-2 dataset. ActLoRA and LoRA consistently achieve a lower loss and higher accuracy compared to

other SOTA methods under our testing conditions. The sweep study in Section 6.3 clearly shows the linear scaling of compute and memory for ActLoRA.

Taken together, our results show that both training and inference costs decrease as the number of adapted layers is reduced. The trainable parameter count and loss reduction is not linearly scalable [15]. This highlights the need for the cautious and targeted allocation of adaptation capacity, where modest reductions in accuracy can be justified by considerable savings in compute, memory, and latency.

7.2. Understanding Optimal K Choice Selection

In order to effectively use this selection technique, it is necessary to select a good value of k without expensive sweep studies. Figure 8 shows the layer-wise activation magnitude. Table 8 and Figure 9 represent the cumulative importance threshold mapping. This data is collected from the sweep study performed in Section 6.3.

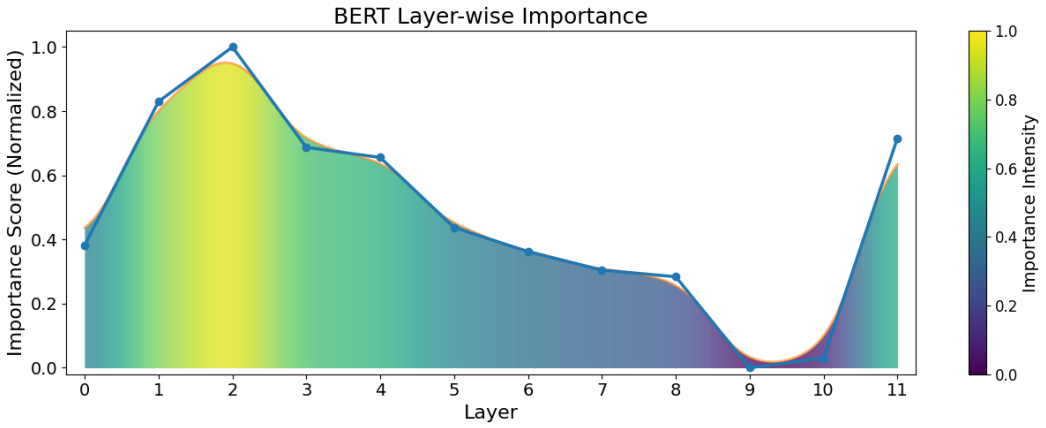


Figure 8. BERT layer importance visualization.

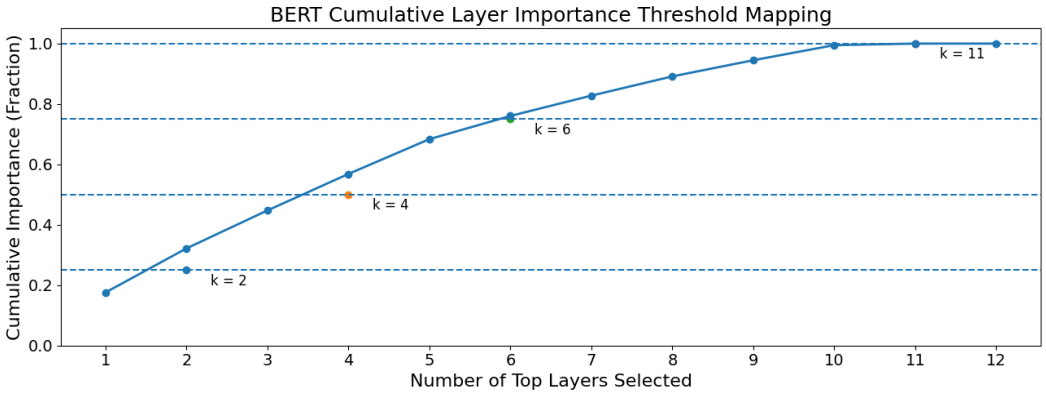


Figure 9. Cummulative Layer Importance Threshold Mapping for BERT.

Table 8. Cumulative importance threshold analysis for BERT layer-wise activation magnitude. The table shows the minimum number of top-ranked layers required to reach each cumulative importance level.

Cumulative Importance Threshold	Top-*k* Layers Required
25%	2
50%	4
75%	6
100%	11

Cumulative Importance Threshold Mapping

Let L denote the total number of layers, and let $\{s_\ell\}_{\ell=1}^{L}$ represent the normalized importance scores obtained from the probe. Without loss of generality, we sort the layers in descending order of importance as follows:

$$s_{(1)} \geq s_{(2)} \geq \cdots \geq s_{(L)}. \tag{40}$$

The cumulative importance function is then defined as

$$C(k) = \frac{\sum_{i=1}^{k} s_{(i)}}{\sum_{i=1}^{L} s_{(i)}}, \quad k = 1, 2, \ldots, L. \tag{41}$$

By construction, $C(k) \in [0, 1]$ and is monotonically increasing with respect to k. For a desired importance threshold $\tau \in (0, 1]$, the minimum number of layers required to retain at least τ fraction of total importance is

$$k_\tau = \min\{k \mid C(k) \geq \tau\}. \tag{42}$$

The cumulative importance analysis reveals a strong concentration of representational energy in a small subset of layers. Only two layers account for roughly 25% of the total magnitude, and four layers already capture about 50%. By six layers, nearly 75% of the cumulative importance is covered, indicating that most of the structural contribution is front-loaded among the highest-ranked layers.

Beyond this point, the curve begins to flatten. Moving from six to 11 layers yields progressively smaller gains. This suggests that the model's functional capacity is not uniformly distributed across depth; instead, a relatively small number of layers dominate the importance mass.

Combining these results with the results from Section 6.3, we can see that, k = 6 provides the best results in terms of accuracy, loss, GPUh and memory consumption. For k < 6, the accuracy is low and loss is higher, whereas for k > 6, the compute and memory does not translate into proportionate gains. Importantly, this static cumulative analysis provides a principled and computation-efficient mechanism for identifying a strong initial estimate of k prior to task-specific fine-tuning, thereby reducing the need for exhaustive sweep studies.

A key assumption of activation-guided layer selection is that activation magnitudes vary meaningfully across layers, enabling the identification of a subset with higher representational energy. However, when all or most Transformer layers exhibit approximately similar activation magnitudes, the discriminative signal required for top-k selection diminishes. Therefore, activation-based selection is most effective when there exists clear heterogeneity in layer-wise activation magnitudes. When the activation profile is flat, alternative importance metrics or adaptive selection strategies may be required.

7.3. Understanding the Stability of Activation Norms for Layer Importance Ranking

We studied the activation magnitude and gradient behavior on the DeBERTaV3-Base and LLama-3.1-8B models. Figure 10 shows the results collected from probing the models for 200 optimizer steps on the SST-2 dataset (GLUE Task—Sentiment Analysis) over 10 random seeds. During each mini batch, the forward pass collects the post-attention activation magnitude and the backward pass collects the attention–projection gradient, which are then used to compute the L2 norm and are averaged over 200 steps and min-max scaled to the range 0–1 per seed.

The figure shows almost perfect overlap for the activation norms over all 10 seeds for both model architectures, whereas the gradient norms show variance across seeds. To quantify this, we used the following metrics:

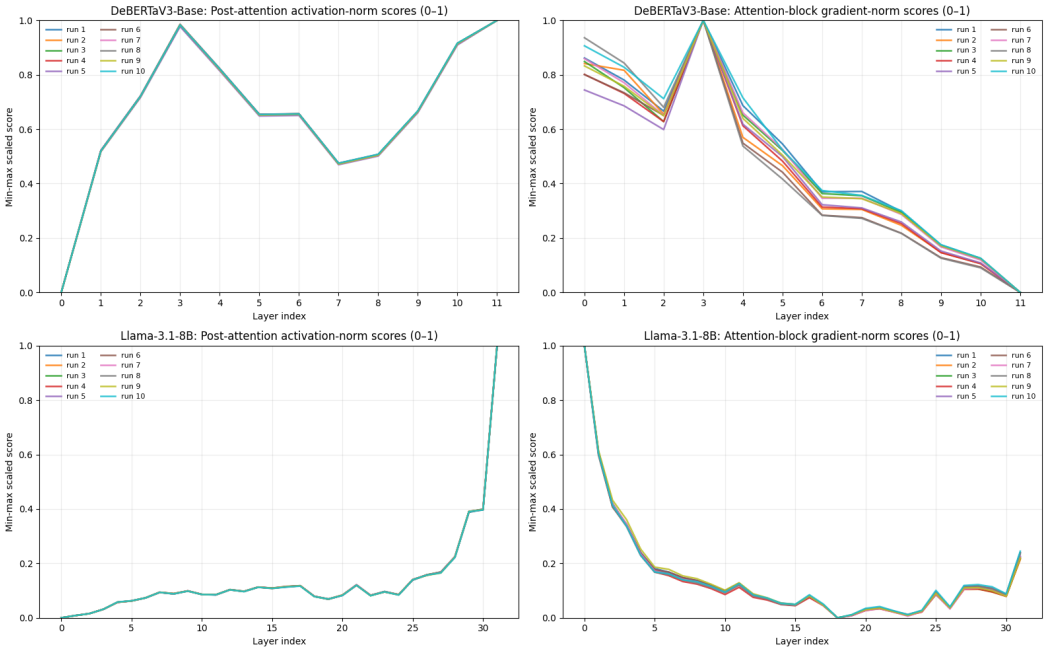


Figure 10. Comparison of activation norm and gradient norm-based layer importance for DeBERTaV3-Base and Llama-3.1-8B across 10 seeds. (**Top**): DeBERTaV3-Base. (**Bottom**): LLama-3.1-8B.

7.3.1. Kendall–τ Ranking Stability

We quantify ranking stability across random seeds using pairwise Kendall–τ correlation between layer importance orderings, refer to Table 9. For DeBERTaV3-Base, activation norms exhibit perfect stability ($\tau = 1.000 \pm 0.000$), indicating identical layer rankings across all seeds. Gradient norms show slightly lower but still high stability ($\tau = 0.939 \pm 0.049$). For LLaMA-3.1-8B, activation norms remain highly stable ($\tau = 0.919 \pm 0.061$), whereas gradient norms display substantially lower agreement ($\tau = 0.638 \pm 0.194$), indicating greater sensitivity to seed variation. Overall, activation-based importance produces more consistent layer rankings across both architectures.

Table 9. Mean and standard deviation of pairwise Kendall–τ rank correlation across 10 seeds.

Model	Metric	Kendall–τ (Mean $\pm$ Std)
DeBERTaV3-Base	Activation Norms	1.0000 ± 0.0000
DeBERTaV3-Base	Gradient Norms	0.9387 ± 0.0489
LLaMA-3.1-8B	Activation Norms	0.9190 ± 0.0608
LLaMA-3.1-8B	Gradient Norms	0.6384 ± 0.1942

7.3.2. Top-k Jaccard Stability

To evaluate selection-level robustness, we compute top-k Jaccard similarity across seeds for $k \in \{2, 4, 6, 8, 10\}$—refer to Table 10. Activation norms yield perfect agreement ($J = 1.0$) for all k and both models, demonstrating fully deterministic top-k layer selection. Gradient norms are also perfectly stable in most settings, with the only deviation observed for DeBERTaV3-Base at $k = 4$ ($J = 0.778 \pm 0.199$). These results indicate that while gradient-based rankings may fluctuate slightly, the resulting top-k subsets remain largely consistent, especially for larger k. In contrast, activation-based selection is completely stable across seeds.

Table 10. Top-*k* Jaccard similarity across 10 seeds.

k	DeBERTa Act	DeBERTa Grad	LLaMA Act	LLaMA Grad
2	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
4	1.0000 ± 0.0000	0.7778 ± 0.1988	1.0000 ± 0.0000	1.0000 ± 0.0000
6	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
8	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
10	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000

This stable behavior of activation norms can be attributed to the underlying substructures developed during the pretraining of the LLM. Since activation magnitudes measure the intrinsic representational structure formed during pretraining [10], they provide a stable signal across sampling variations, whereas gradient norms depend on downstream loss landscapes and optimization dynamics, leading to greater instability. Note that the activation norms may drift once the actual training begins. We are using activation norms statically to determine layer importance before the training starts; we benefit from its stable nature.

7.4. Is Importance Scoring Task-Dependent?

Figure 11 represents different adaptation geometries for DeBERTaV3-Base and LLaMA-3.1-8B, while remaining largely invariant across task types within each model. Across both models, the close alignment of curves across tasks highlights that activation-based layer importance is dominated by models' architectural structure rather than task semantics. We collected two types of activations, pre-attention and post-attention, from both the DeBERTa and Llama models, as depicted in Figure 2. Pre-attention activation norms measure the magnitude of the hidden representations entering the self-attention module and reflect where attention is applied within the network. Post-attention activation norms capture the raw contribution produced by the attention mechanism itself, before residual accumulation and subsequent normalization.

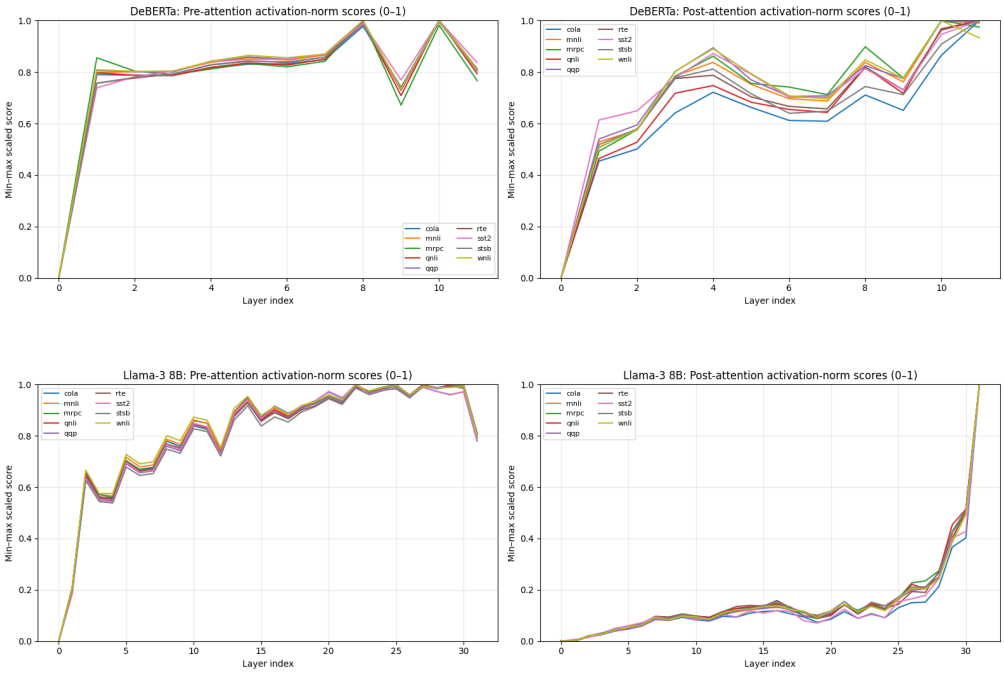


Figure 11. Comparison of layer importance for DeBERTaV3-Base and Llama-3.1-8B across GLUE tasks. (**Top**): DeBERTaV3-Base. (**Bottom**): Llama-3.1-8B.

These results look similar to that of Section 7.3. There is slight task-dependent variation, but the nature of the curves remain more or less similar, thus pointing at a stable intrinsic representational geometry for LLMs. Hence we can say that activation norms are task-independent and are majorly dominated by the backbone model's internal structure, thus answering our RQ-3.

7.5. Ablation Study and Layer Importance Scores

To validate the layer importance scores estimated using activation norms, we compared them against several established metrics including gradient norms, weight delta, Taylor scores, and Fisher information. The ground truth layer importance was inferred from the ablation study performed on the DeBERTaV3-Base model fine-tuned on the SST-2 dataset using LoRA with r = 8 applied to the query (Q) and value (V) projections of the attention blocks. After training the model, we ablated the Q and V projections, i.e, set their weights to zero and in absence of these layers, we captured the accuracy. The inferred layer importance is a direct function of the drop in accuracy with the baseline. The assumption is that if a layer's absence results in a higher drop in accuracy, the layer is assumed to be important. We normalized the absolute values to get the final importance score which is used as an empirical baseline for comparing other importance scores.

The results are visualized as heatmaps in Figure 12. We can observe that gradient norm, Fisher, and Taylor-based scores show similar patterns in both Q and V projections. Weight delta, on the other hand, follows the well-known trend of increasing importance in later layers, likely due to vanishing gradients leading to smaller weight updates in earlier layers or optimizer behavior where weight update is in the opposite direction of gradient.

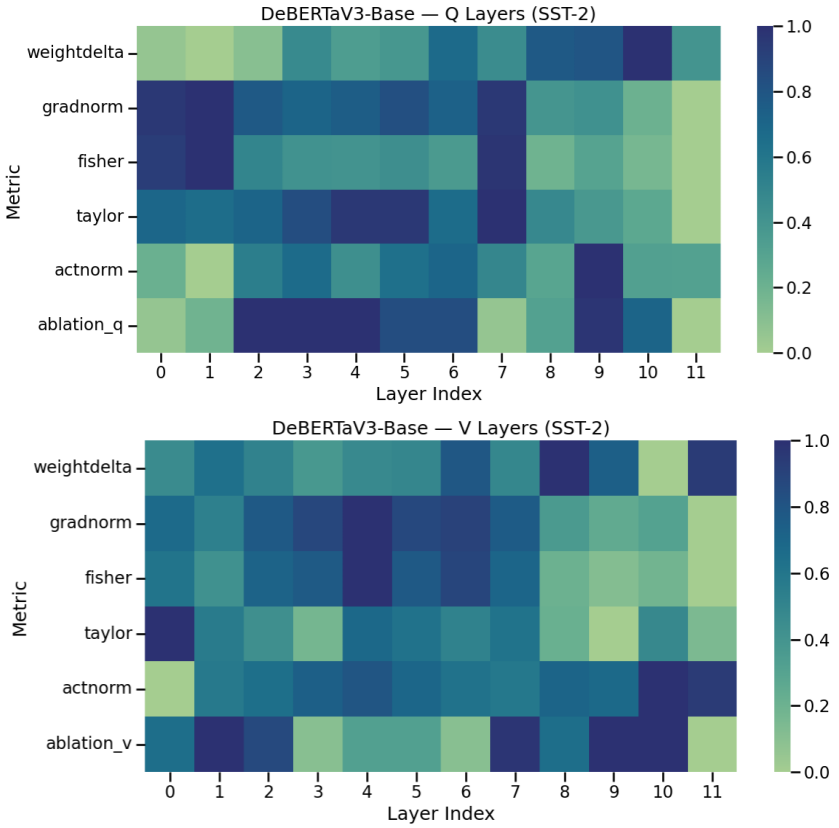


Figure 12. Comparison of layer importance for Q and V projections using various metrics.

Notably, the layer importance estimated by activation norms aligns most closely with the empirical ablation results. To quantify this observation, we computed correlations be-

tween all metrics and the ablation-based importance, as shown in Table 11. Using Pearson's, Spearman's, Kendall's, and cosine similarity measures, the results consistently show that activation norms achieve the highest correlation with ablation-based importance across both Q and V projections, followed by Taylor-based scores. Although the absolute correlation magnitudes are moderate, the consistent relatively higher correlation of activation norms with ablation results indicates that they are more predictive of the functional layer importance compared to the gradient-based, weight-based or curvature-based methods.

Table 11. Correlation between importance metrics and ablation performance for Query (Q) and Value (V) layers using Pearson's (ρ), Spearman's (ρ_s), Kendall's (τ), and cosine similarity ($\cos(\theta)$).

Layer	Metric	ρ	ρ_s	τ	$\cos(\theta)$
Query	actnorm	0.18	0.65	0.46	0.16
	Taylor	0.15	0.32	0.18	0.14
	gradnorm	0.15	−0.063	0.00	0.14
	Fisher	−0.00051	0.00	0.092	−0.00046
	weightdelta	−0.33	0.11	0.031	−0.29
Value	actnorm	0.34	−0.25	−0.23	0.28
	Taylor	−0.03	0.067	0.078	−0.025
	gradnorm	−0.38	−0.39	−0.36	−0.32
	Fisher	−0.44	−0.38	−0.33	−0.36
	weightdelta	−0.60	−0.23	−0.14	−0.50

8. Limitations and Future Works

This study has certain limitations. First, while methods such as LoRA, AdaLoRA, and DyLoRA primarily focus on maximizing task accuracy, they do not expose standardized metrics for evaluating efficiency in terms of latency, GPU utilization, memory consumption, or throughput. As a result, direct and comprehensive efficiency comparisons are difficult. Within the scope of our resources, we compared LoRA and AdaLoRA against the GLUE bench rigorously, but due to resource limitations, we could not perform a SOTA PEFT comparison on a larger scale.

Second, due to compute constraints, we were unable to scale experiments to very large models (e.g., 70 B–80 B parameters), which may exhibit different adaptation dynamics especially when selecting k.

Thirdly, we studied the behavior of post-attention activation norms, but activations are generated at each layer of an encoder/decoder architecture. Hence these activation vectors remain open to exploration. Moreover, we only applied LoRA adapters to query and value projections; further investigations into the benefits of selective LoRA adapters for other modules can be explored.

Lastly, we can exploit the structural properties of pretrained models for selective fine-tuning as long as they show variation in activation magnitude. If the layer-wise importance curve remains largely flat, then activation-guided selective fine-tuning may not provide the best results, thus making room for more robust metrics.

9. Conclusions

In this work, we explored a simple extension of Low-Rank Adaptation in which LoRA is applied to only a subset of Transformer layers. We also showed how activation norms influence the LoRA updates. Our results suggest that adapting a limited number of layers can retain a competitive performance while reducing the number of trainable parameters and associated training costs.

We also observed that activation magnitudes are stable and represent the model's internal geometry. While preliminary, these findings indicate that activation norms provide a practical heuristic for guiding sparse adaptation.

Finally, because our method builds directly on LoRA, it is immediately compatible with QLoRA, ZeRO, DDP, and FSDP, making it straightforward to deploy in distributed or memory-constrained training settings.

Author Contributions: Conceptualization, A.D., P.S., R.V.B. and N.S.; methodology, A.D.; software, A.D.; validation, A.D., P.S., R.V.B. and N.S.; formal analysis, A.D.; investigation, A.D.; data curation, A.D.; writing—original draft preparation, A.D.; writing—review and editing, A.D., P.S. and R.V.B.; supervision, P.S., R.V.B. and N.S.; project administration, N.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original data presented in this study are openly available as part of the General Language Understanding Evaluation (GLUE) benchmark and can be accessed at https://gluebenchmark.com (accessed on 31 January 2026).

Conflicts of Interest: Author Pooja Shyamsundar was employed by the company IBM. Author Rashmi Vishwanath Bhat was employed by the company Salesforce. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention is all you need. *arXiv* **2017**, arXiv:1706.03762.
2. Wolf, T.; Debut, L.; Sanh, V.; Chaumond, J.; Delangue, C.; Moi, A.; Cistac, P.; Rault, T.; Louf, R.; Funtowicz, M.; et al. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Online, 16–20 November 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 38–45.
3. Sanh, V.; Debut, L.; Chaumond, J.; Wolf, T. DistilBERT: A distilled version of BERT. *arXiv* **2019**, arXiv:1910.01108.
4. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv* **2019**, arXiv:1810.04805. [CrossRef]
5. Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; de Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; Gelly, S. Parameter-Efficient Transfer Learning for NLP. *arXiv* **2019**, arXiv:1902.00751. [CrossRef]
6. Hu, E.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W. LoRA: Low-rank adaptation of large language models. In Proceedings of the International Conference on Learning Representations (ICLR), Online, 25–29 April 2022. Available online: https://openreview.net/forum?id=nZeVKeeFYf9 (accessed on 20 January 2026).
7. Liu, H.; Tamkin, A.; Hajishirzi, M.; Smith, N.A. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. *arXiv* **2022**, arXiv:2205.05638.
8. Lester, B.; Al-Rfou, R.; Constant, N. The Power of Scale for Parameter-Efficient Prompt Tuning. *arXiv* **2021**, arXiv:2104.08691. [CrossRef]
9. Li, J.; Liang, Y. Prefix-Tuning: Optimizing Continuous Prompts for Generation. *arXiv* **2021**, arXiv:2101.00190. [CrossRef]
10. Wang, Q.; Shen, S. Activation-guided low-rank parameter adaptation for efficient model fine-tuning. *IEEE Access* **2025**, *13*, 70909–70918. [CrossRef]
11. Meta AI. The LLaMA 3 Herd of Models: Open and Efficient Foundation Language Models. April 2024. Available online: https://ai.meta.com/llama/ (accessed on 20 January 2026).
12. Voita, E.; Talbot, D.; Moiseev, F.; Sennrich, D.; Titov, I. Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting. *arXiv* **2019**, arXiv:1905.09418.
13. Zhang, R.; Han, S.; Gao, H.; Zhang, W.; Liu, S. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In Proceedings of the International Conference on Learning Representations (ICLR), Online, 1–5 May 2023. Available online: https://openreview.net/forum?id=lq62uWRJjiY (accessed on 20 January 2026).

14. Liao, X.; Wang, C.; Zhou, S.; Hu, J.; Zheng, H.; Gao, J. Dynamic Adaptation of LoRA Fine-Tuning for Efficient and Task-Specific Optimization of Large Language Models. *arXiv* **2025**, arXiv:2501.14859.
15. Kaplan, J.; McCandlish, S.; Henighan, T.; Brown, T.B.; Chess, B.; Child, R.; Gray, S.; Radford, A.; Wu, J.; Amodei, D. Scaling laws for neural language models. *arXiv* **2020**, arXiv:2001.08361. [CrossRef]
16. Hoffmann, J.; Borgeaud, S.; Mensch, A.; Buchatskaya, E.; Cai, T.; Rutherford, E.; Casas, D.d.; Hendricks, L.; Welbl, J.; Rings, F.; et al. Training compute-optimal large language models. *arXiv* **2020**, arXiv:2203.15556.
17. Sajjad, H.; Dalvi, F.; Durrani, N.; Nakov, P. On the Effect of Dropping Layers of Pre-trained Transformer Models. *Comput. Speech Lang.* **2023**, *77*, 101429. [CrossRef]
18. Fan, A.; Grave, E.; Joulin, A. Reducing Transformer Depth on Demand with Structured Dropout. In Proceedings of the International Conference on Learning Representations (ICLR), Addis Ababa, Ethiopia, 26–30 April 2020.
19. Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv* **2019**, arXiv:1907.11692.
20. Yao, K.; Gao, P.; Li, L.; Zhao, Y.; Wang, X.; Wang, W.; Zhu, J. Layer-wise Importance Matters: Less Memory for Better Performance in Parameter-efficient Fine-tuning of Large Language Models. *arXiv* **2020**, arXiv:2410.11772.
21. Ke, W.; Wang, J.; Wang, P.; Liu, J.; Nie, D.; Li, G.; Li, Y. Unveiling LoRA intrinsic ranks via salience analysis. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 131575–131595.
22. Chen, H.; Garner, P.N. A Bayesian interpretation of adaptive low-rank adaptation. *arXiv* **2024**, arXiv:2409.10673. [CrossRef]
23. Valipour, M.; Rezagholizadeh, M.; Kobyzev, I.; Ghodsi, A. DyLoRA: Parameter-efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, Dubrovnik, Croatia, 2–6 May 2023*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 3274–3287.
24. Rippel, O.; Jebara, T. Learning ordered representations with nested dropout. *Proc. ICML (PMLR)* **2014**, *32*, 1746–1754.
25. Liang, J.; Liu, C.; Yu, Y.; Xu, Y. ALoRA: Allocating low-rank adaptation for fine-tuning large language models. *arXiv* **2024**, arXiv:2403.16187.
26. Xu, H.; Xu, H.; Chen, L.; Kong, L. AutoLoRA: Automatically tuning matrix ranks in low-rank adaptation based on meta learning. *arXiv* **2024**, arXiv:2403.09113.
27. Chen, H.; Zhang, J.; Chen, Y. LoRA-drop: Efficient LoRA parameter pruning based on output evaluation. *arXiv* **2024**, arXiv:2402.07721.
28. Yang, A.; Chen, L.; Liu, Z.; Tang, J. LoRA-FA: Memory-efficient low-rank adaptation for large language model fine-tuning. *arXiv* **2023**, arXiv:2308.03303.
29. Zhao, J.; Ren, Z.; Zhao, K.; Ma, R.; Wu, J.; Huang, H. GaLore: Memory-efficient LLM training by gradient low-rank projection. In Proceedings of the International Conference on Machine Learning (ICML), Online, 7–11 May 2024. Available online: https://openreview.net/forum?id=hYHsrKDiX7 (accessed on 20 January 2026).
30. Dettmers, T.; Pagnoni, A.; Holtzman, A.; Zettlemoyer, L. QLoRA: Efficient fine-tuning of quantized large language models. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), New Orleans, LA, USA, 10–16 December 2023.
31. Meng, C.; Deng, C.; Shen, Y.; Yang, H.; Zhang, Y. LoRA+: Efficient low-rank adaptation of large models. *arXiv* **2024**, arXiv:2402.12354.
32. Buehler, E.L.; Buehler, M.J. X-LoRA: Mixture of low-rank adapter experts, a flexible framework for large language models with applications in protein mechanics and molecular design. *Apl. Mach. Learn.* **2024**, *2*, 026119. [CrossRef]
33. Zhang, Z.; Hu, Y.; Li, X.; Zhao, J. Delta-LoRA: Fine-tuning high-rank parameters with the delta of low-rank matrices. *arXiv* **2023**, arXiv:2309.02411.
34. Liu, Z.; Xu, H.; Wang, X.; Yu, Y. DoRA: Weight-decomposed low-rank adaptation. *arXiv* **2024**, arXiv:2402.09353.
35. Kopiczko, R.; Tjandra, M.; Simonyan, K. VeRA: Vector-based random matrix adaptation. *arXiv* **2023**, arXiv:2310.11454.
36. Li, L.; Lin, C.; Li, D.; Huang, Y.-L.; Li, W.; Wu, T.; Zou, J.; Xue, W.; Han, S.; Guo, Y. Efficient Fine-Tuning of Large Models via Nested Low-Rank Adaptation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), Honolulu, HI, USA, 19–23 October 2025*; IEEE: New York, NY, USA, 2025; pp. 22252–22262.
37. Hyeon-Woo, N.; Moon, Y.-B.; Oh, T.-H. FedPara: Low-Rank Hadamard Product for Communication-Efficient Federated Learning. *arXiv* **2021**, arXiv:2108.06098.
38. Yeh, S.-Y.; Hsieh, Y.-G.; Gao, Z.; Yang, B.B.W.; Oh, G.; Gong, Y. Navigating Text-To-Image Customization: From LyCORIS Fine-Tuning to Model Evaluation. *arXiv* **2023**, arXiv:2309.14859.
39. [CrossRef] Kirkpatrick, J.; Pascanu, R.; Rabinowitz, N.; Veness, J.; Desjardins, G.; Rusu, A.A.; Milan, K.; Quan, J.; Ramalho, T.; Grabska-Barwinska, A.; et al. Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci. USA* **2017**, *114*, 3521–3526. [CrossRef]

40. Tenney, I.; Das, D.; Pavlick, E. BERT Rediscovers the Classical NLP Pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019; pp. 4593–4601.
41. Michel, M.; Levy, O.; Neubig, G. Are Sixteen Heads Really Better than One? *arXiv* **2019**, arXiv:1905.10650. [CrossRef]
42. Hao, Y.; Zeng, A.; Sakti, S. Self-Attention Attribution: Interpreting Information Interactions Inside Transformer. *arXiv* **2020**, arXiv:2004.11207. [CrossRef]
43. Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. Multi-Head or Single-Head? An Empirical Comparison. *arXiv* **2021**, arXiv:2106.09650.
44. LeCun, Y.; Denker, J.S.; Solla, S.A. Optimal brain damage. In Proceedings of the NIPS'89: The 3rd International Conference on Neural Information Processing Systems, San Diego, CA, USA, 2–7 December 1990.
45. Hassibi, B.; Stork, D.G. Second-order derivatives for network pruning: Optimal brain surgeon. In *Proceedings of the Advances in Neural Information Processing Systems 5 (NIPS 1992), San Francisco, CA, USA, 30 November–3 December 1992*; Morgan Kaufmann Publishers Inc.: San Francisco, CA, USA, 1993.
46. Theis, L.; Korshunova, I.; Tejani, A.; Huszár, F. Faster gaze prediction with dense networks and Fisher pruning. *arXiv* **2018**, arXiv:1801.05787. [CrossRef]
47. Li, H.; Kadav, A.; Durdanovic, I.; Samet, H.; Graf, H.P. Pruning filters for efficient ConvNets. *arXiv* **2017**, arXiv:1608.08710. [CrossRef]
48. Olsson, K.; Weller, A.; Rocktäschel, T. In-Context Learning and Induction Heads. *arXiv* **2022**, arXiv:2202.01046. [CrossRef]
49. Xu, Y.; Liang, Y.; Dai, S.; Hu, T.; Chan, T.N.; Ma, C. Understanding and Guiding Layer Placement in Parameter-Efficient Fine-Tuning of Large Language Models. *arXiv* **2020**, arXiv:2602.04019.
50. Wang, A.; Singh, A.; Michael, J.; Hill, F.; Levy, O.; Bowman, S.R. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *arXiv* **2019**, arXiv:1804.07461. [CrossRef]
51. He, P.; Liu, X.; Gao, J.; Chen, W. DeBERTa: Decoding-enhanced BERT with disentangled attention. *arXiv* **2021**, arXiv:2006.03654.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.